Design Pattern

A Design Pattern is a proven, reusable solution to a commonly occurring problem in software design.

ARCHITECTURAL PATTERNS

How is the whole system structured?

Examples: Layered Architecture · MVC

→

DESIGN PATTERNS

How do classes and objects work together?

Examples: Factory · Observer · Strategy

Design Pattern

A Design Pattern is a proven, reusable solution to a commonly occurring problem in software design.

ARCHITECTURAL PATTERNS

How is the whole system structured?

Examples: Layered Architecture · MVC

→

DESIGN PATTERNS

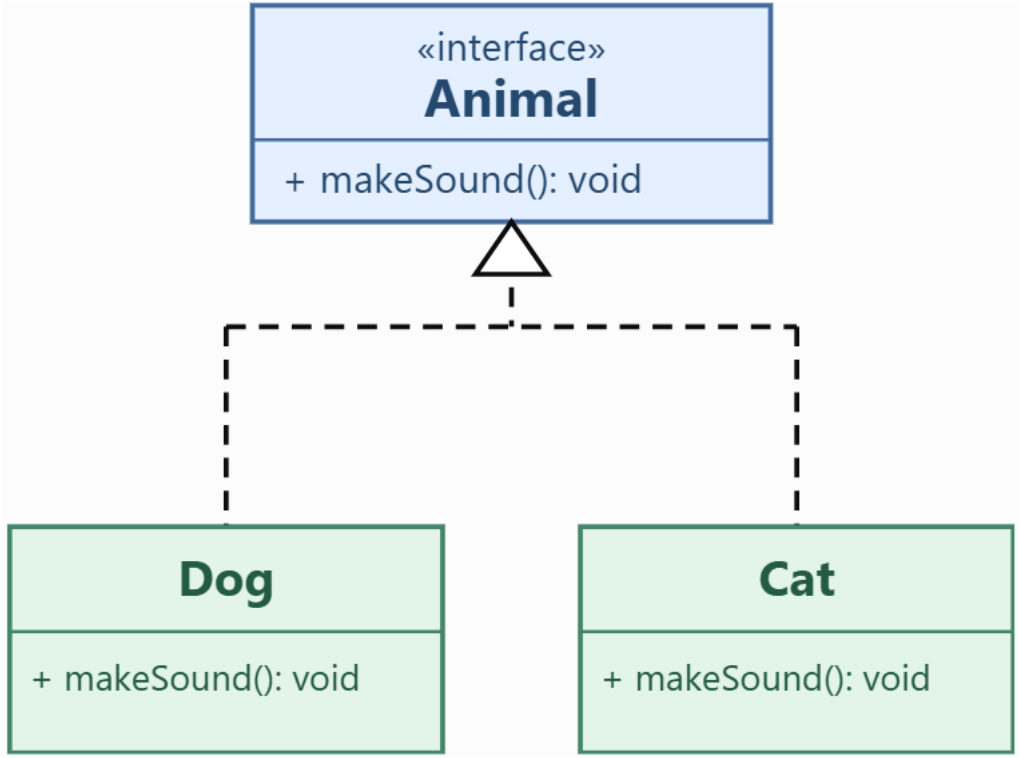
How do classes and objects work together?

Examples: Factory · Observer · Strategy

Interface

An interface is a contract that defines what a class must do, but not how it does it.

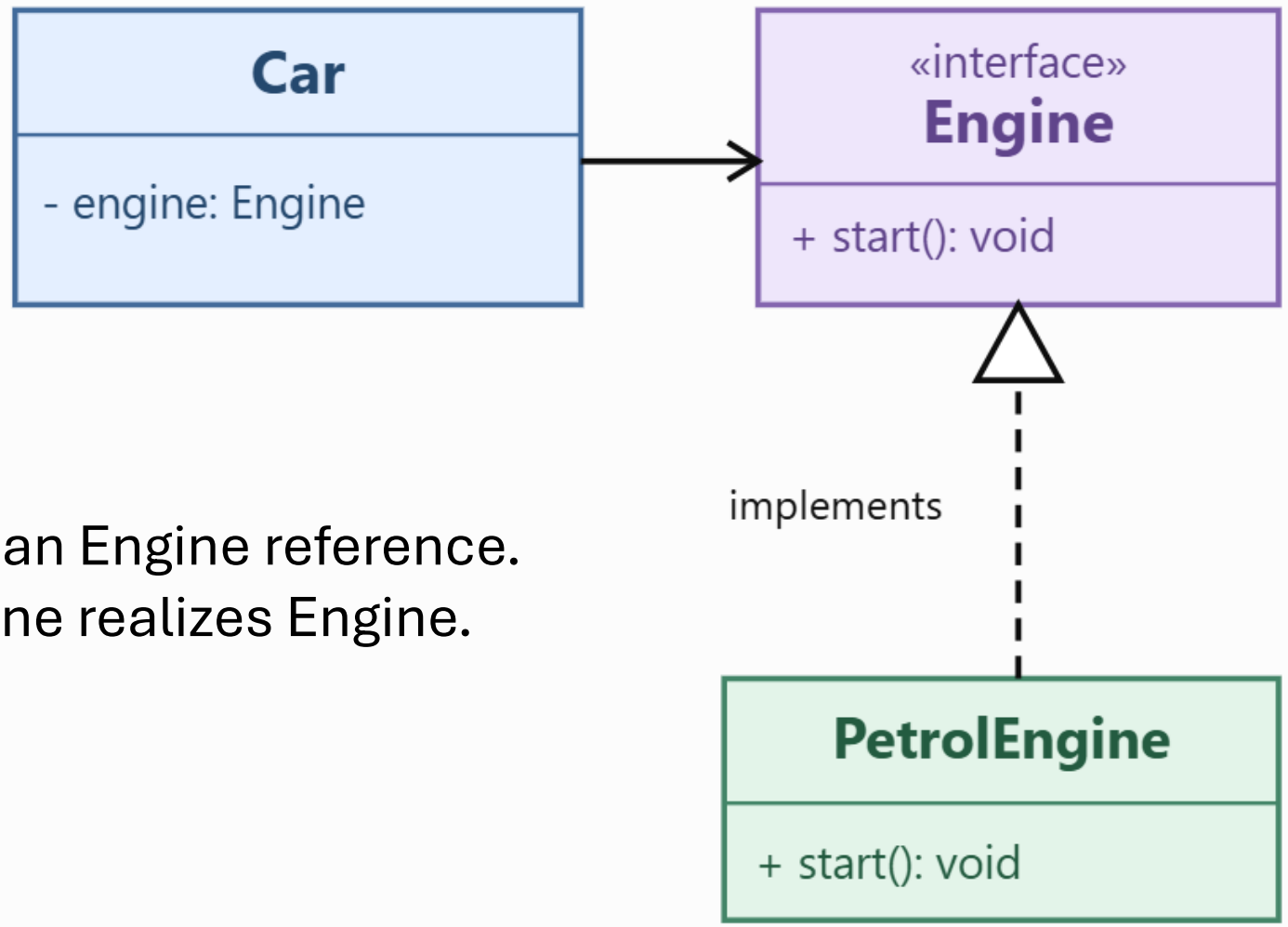
- An interface declares methods.
- A class implements the interface by providing the methods' behavior.
- Different classes can implement the same interface in different ways.



Interface

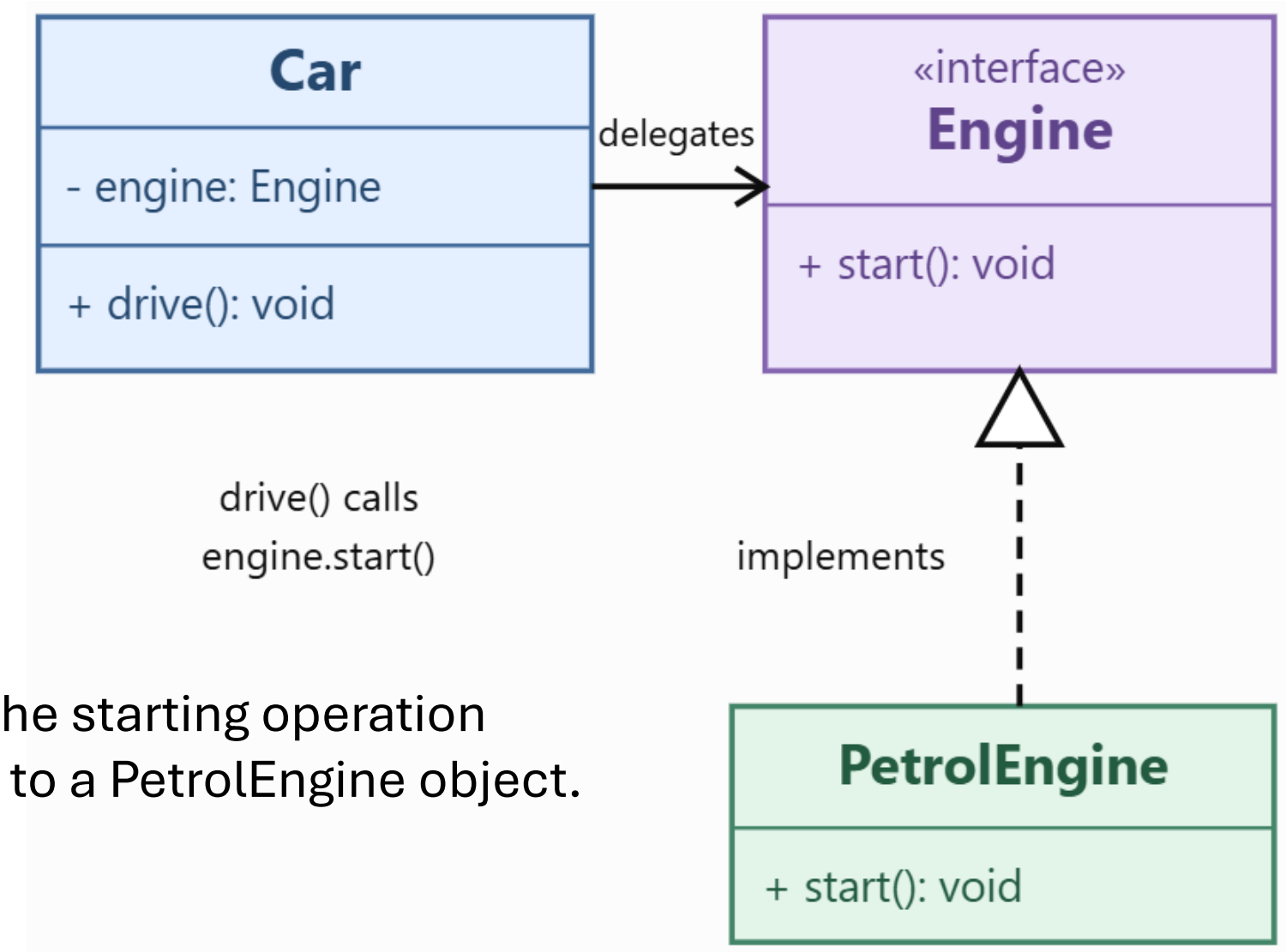


Reference to Interface



Car holds an Engine reference.
PetrolEngine realizes Engine.

Delegation Through Interface

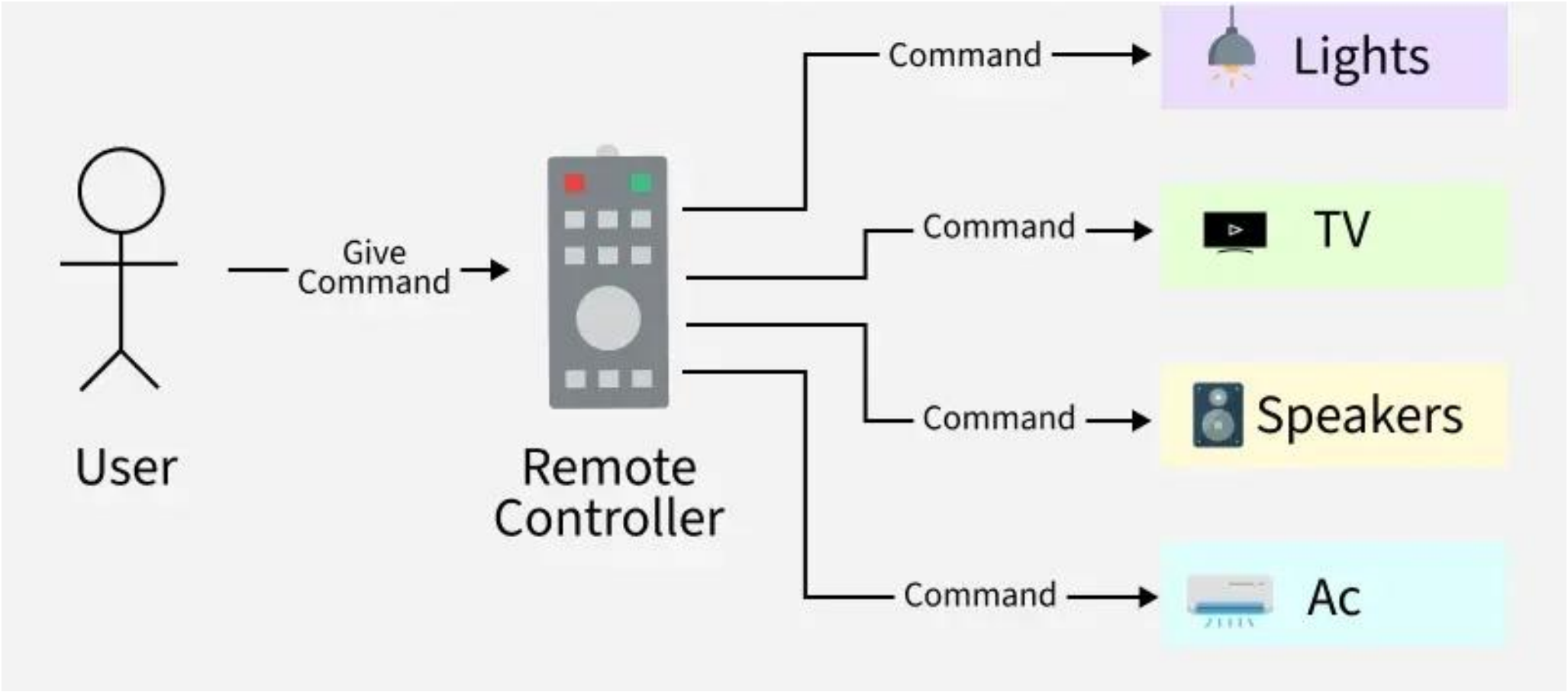


Car delegates the starting operation through Engine to a PetrolEngine object.

Command Pattern

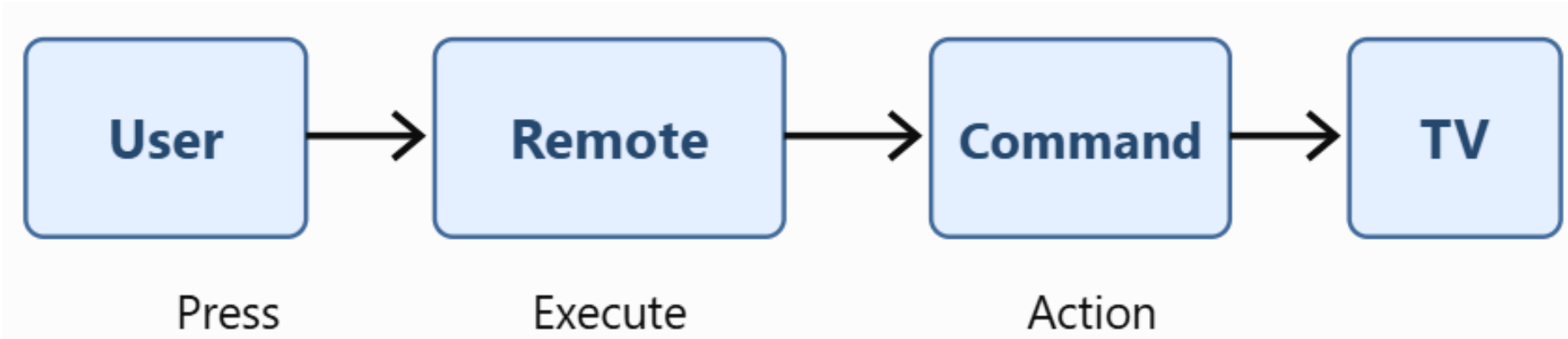


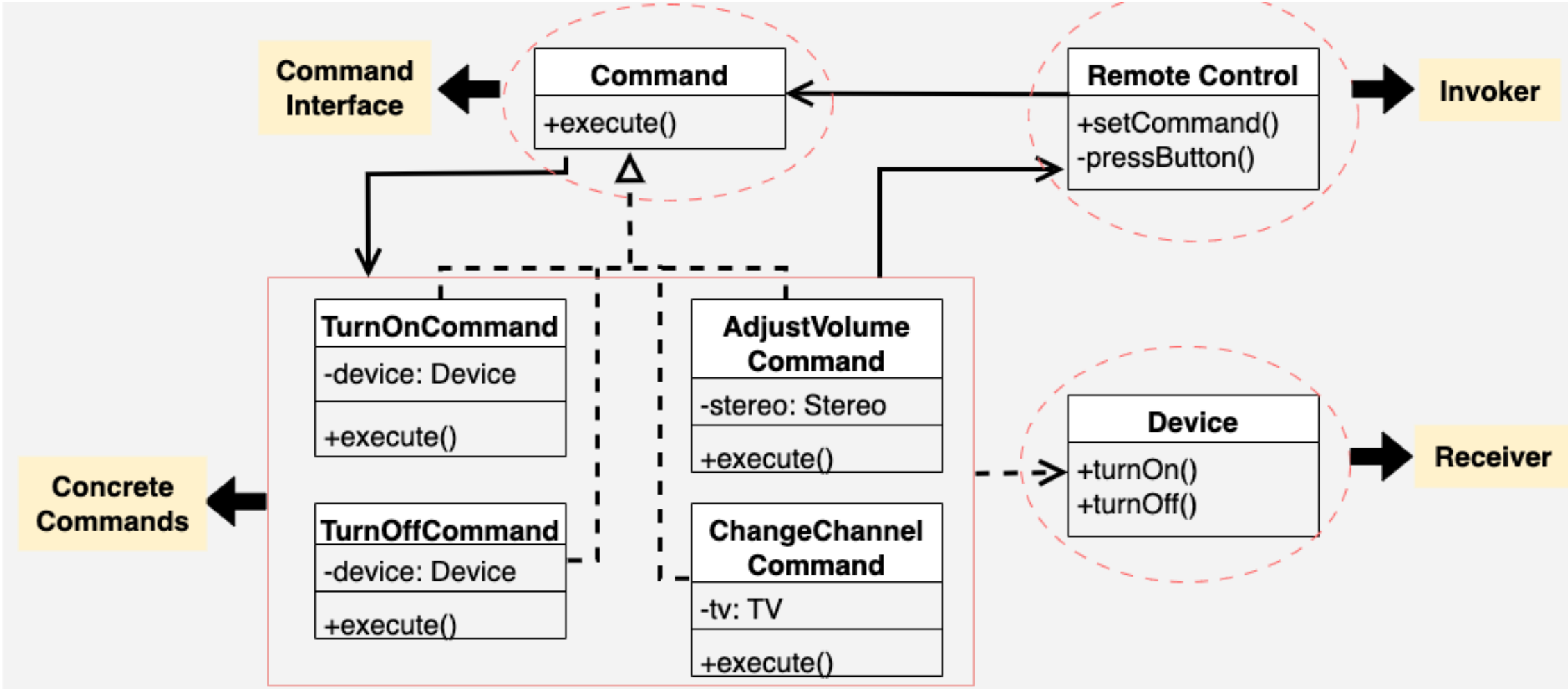
Command Pattern

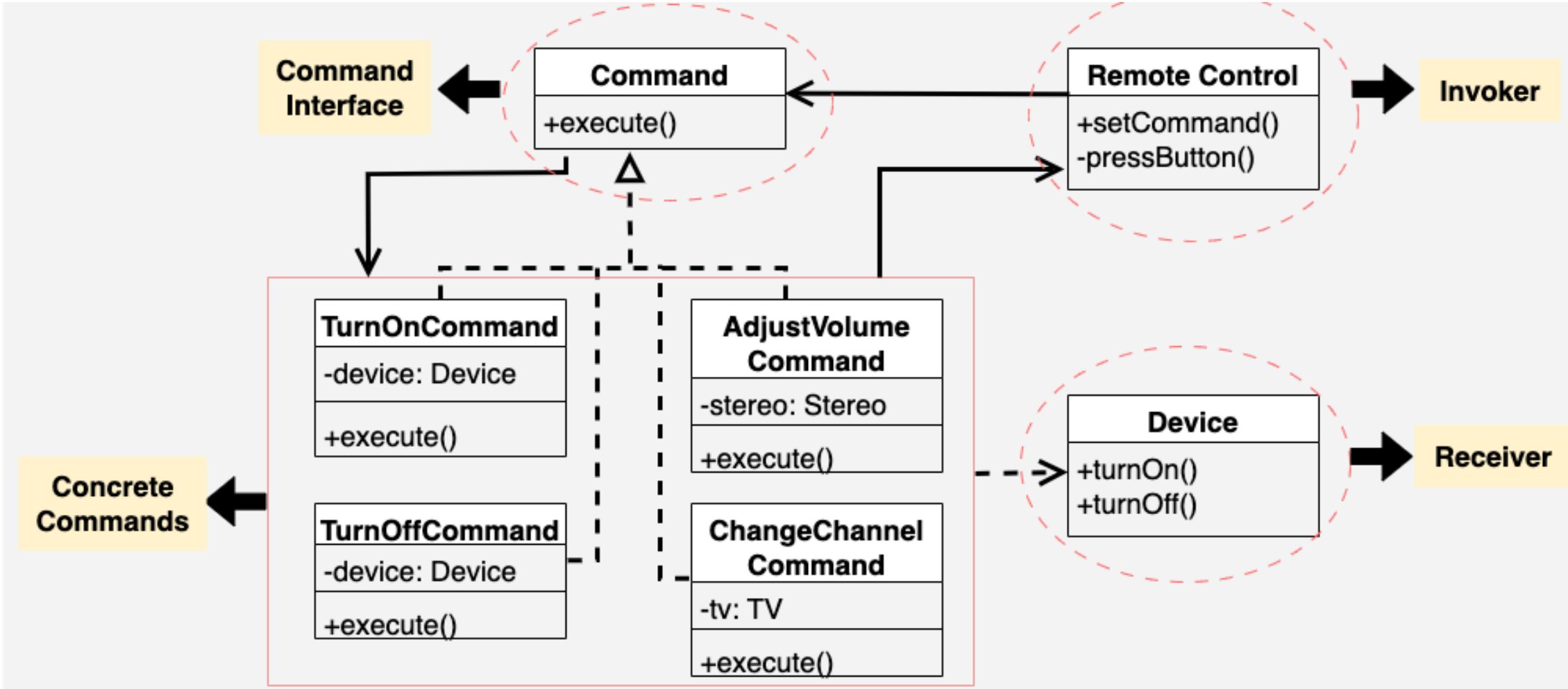


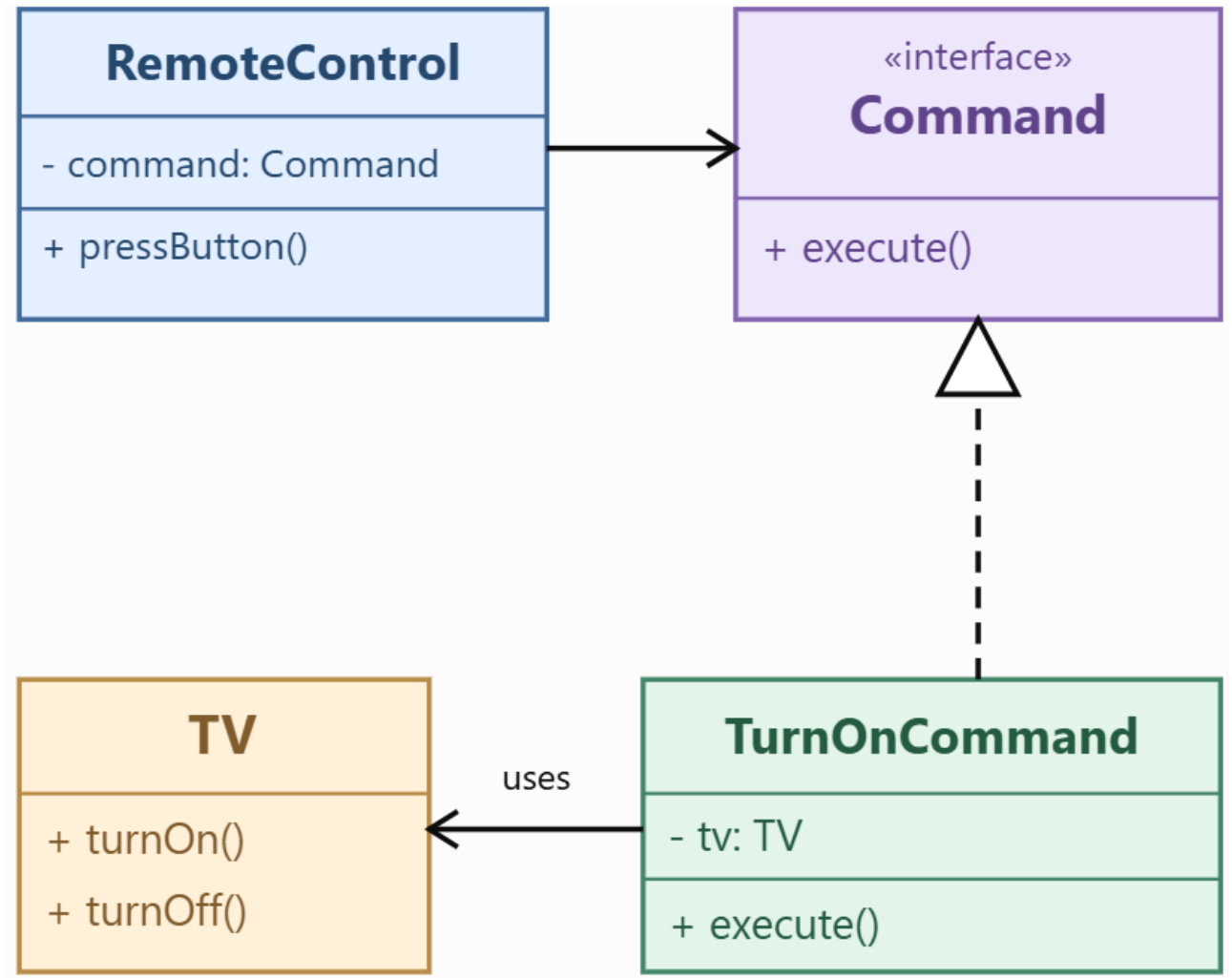
Instead of telling a device directly what to do, the remote asks a command object to perform the action.

Source: geeksforgeeks.org/system-design/command-pattern


User
Remote
Command
TV
Press
Execute
Action


Command Interface
Command
+execute()
Remote Control
+setCommand()
-pressButton()
Invoker
TurnOnCommand
-device: Device
+execute()
AdjustVolume Command
-stereo: Stereo
+execute()
Concrete Commands
TurnOffCommand
-device: Device
+execute()
ChangeChannel Command
-tv: TV
+execute()
Device
+turnOn()
+turnOff()
Receiver


Command Interface
Command
+execute()
Remote Control
+setCommand()
-pressButton()
Invoker
TurnOnCommand
-device: Device
+execute()
AdjustVolume Command
-stereo: Stereo
+execute()
Concrete Commands
TurnOffCommand
-device: Device
+execute()
ChangeChannel Command
-tv: TV
+execute()
Device
+turnOn()
+turnOff()
Receiver


RemoteControl
- command: Command
+ pressButton()
«interface»
Command
+ execute()
TV
+ turnOn()
+ turnOff()
uses
TurnOnCommand
- tv: TV
+ execute()

Command Pattern

When to Use

- **Decoupling:** When you want to keep the sender (like a button) separate from the receiver (like a TV) to make things more flexible.
- **Undo/Redo:** When you need to support reversing actions (e.g., undoing a delete).
- **Queues/Logging:** When you want to keep a history of commands, log them, or run them later in order.
- **Dynamic Configuration:** When you want to assemble and swap commands at runtime without changing the main code.

Command Pattern

When Not to Use

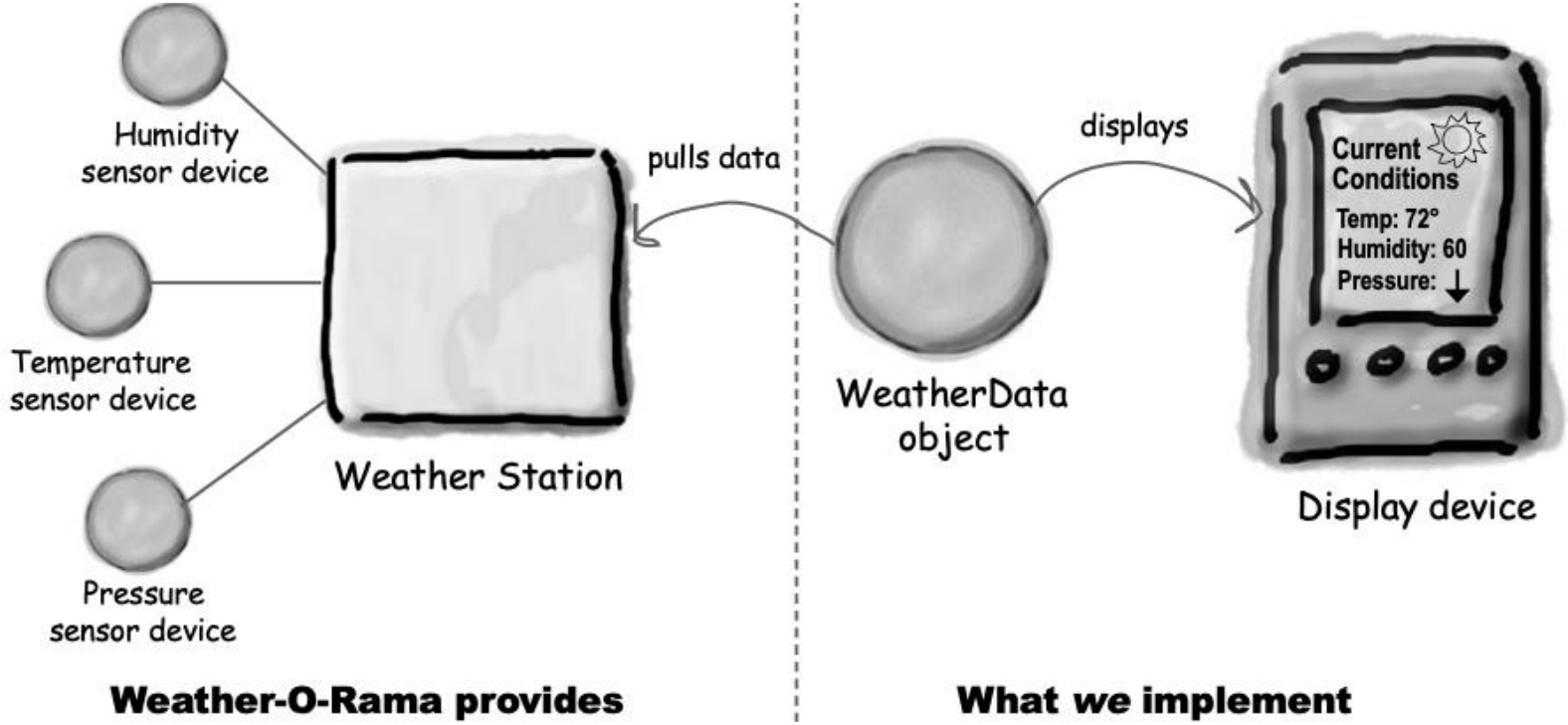
- **Simple Tasks:** If the job is very small and will never be reused, this pattern is overkill.
- **Tight Coupling is Fine:** If it's okay for the sender to directly call the receiver, you don't need this extra layer.
- **Performance Critical:** Avoid if even small overhead is a problem — the pattern adds extra objects and calls.
- **No Undo/Redo Needed:** If you don't need to reverse actions, simpler code is better.

Observer Pattern



The Weather Monitoring app problem

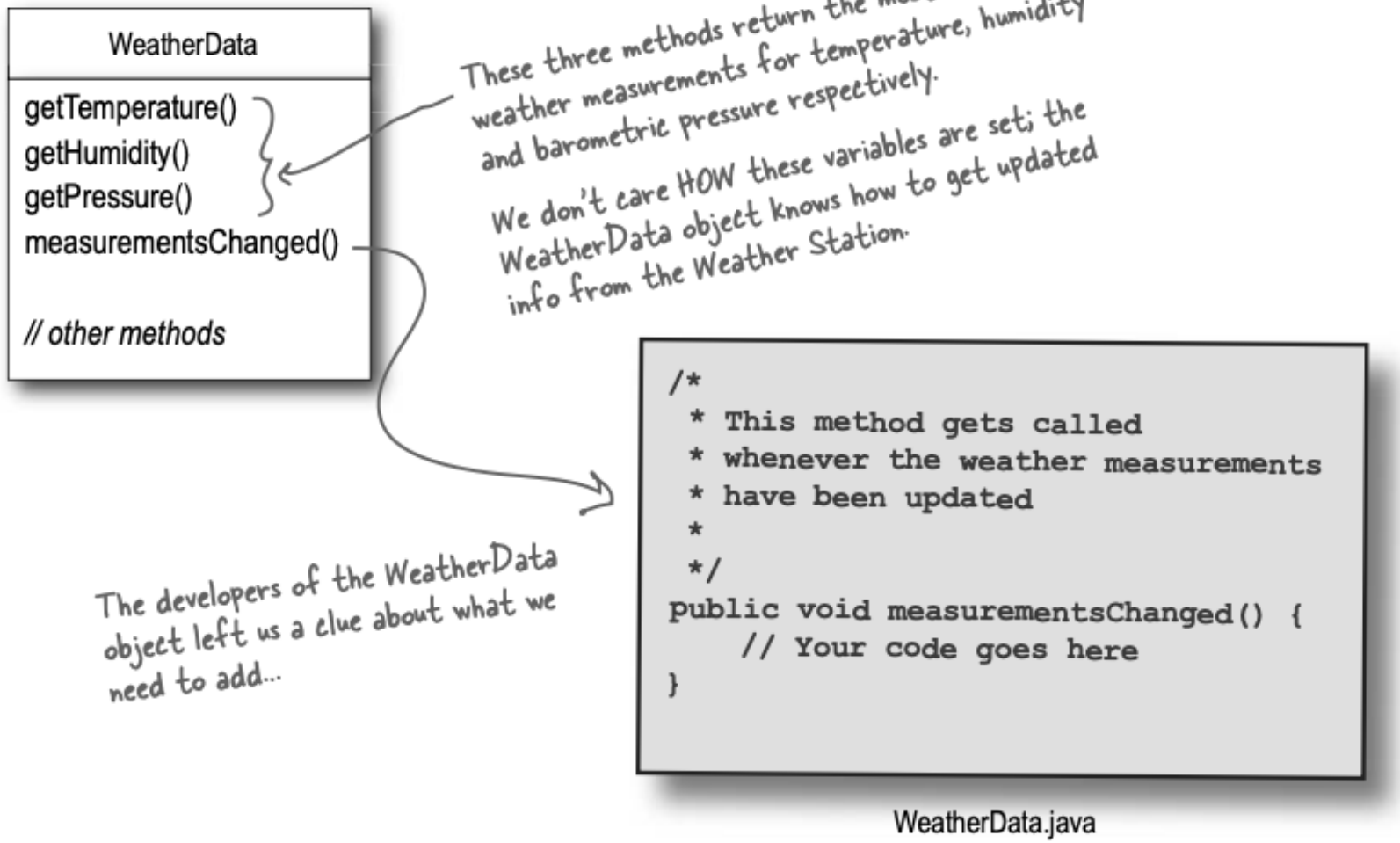
The three players in the system are the weather station (the physical device that acquires the actual weather data), the WeatherData object (that tracks the data coming from the Weather Station and updates the displays), and the display that shows users the current weather conditions.



The Weather Monitoring app problem

The WeatherData object knows how to talk to the physical Weather Station.

Our job is to implement measurementsChanged() so that it updates the display device(s) for current conditions, weather stats, and forecast.

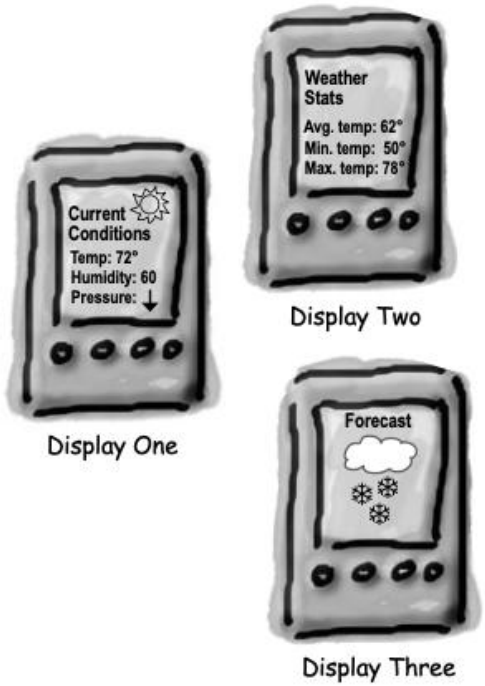


The Weather Monitoring app problem

Suppose we have three display devices to update, here's a first implementation possibility -- we'll add our code to the measurementsChanged() method:

```
public class WeatherData {
        // instance variable declarations
        public void measurementsChanged() {
                float temp = getTemperature();
                float humidity = getHumidity();
                float pressure = getPressure();

                DisplayOne.update(temp, humidity, pressure);
                DisplayTwo.update(temp, humidity, pressure);
                DisplayThree.update(temp, humidity, pressure);
        }
        // other WeatherData methods here
}
```



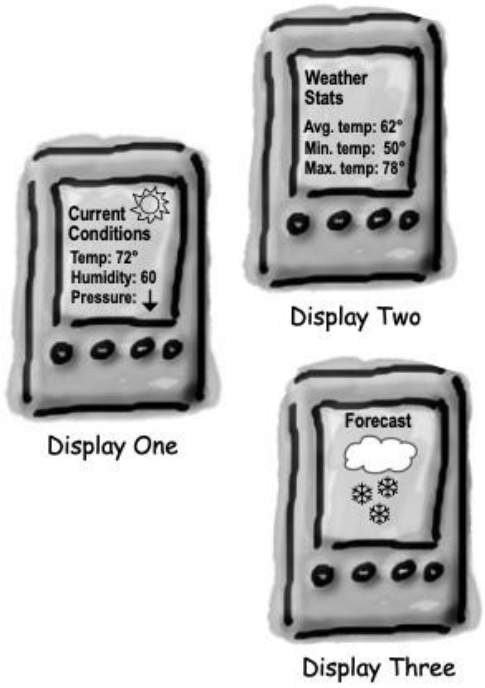
The Weather Monitoring app problem

Suppose we have three display devices to update, here's a first implementation possibility -- we'll add our code to the measurementsChanged() method:

```
public class WeatherData {
        // instance variable declarations
        public void measurementsChanged() {
                float temp = getTemperature();
                float humidity = getHumidity();
                float pressure = getPressure();

                DisplayOne.update(temp, humidity, pressure);
                DisplayTwo.update(temp, humidity, pressure);
                DisplayThree.update(temp, humidity, pressure);
        }
        // other WeatherData methods here
}
```

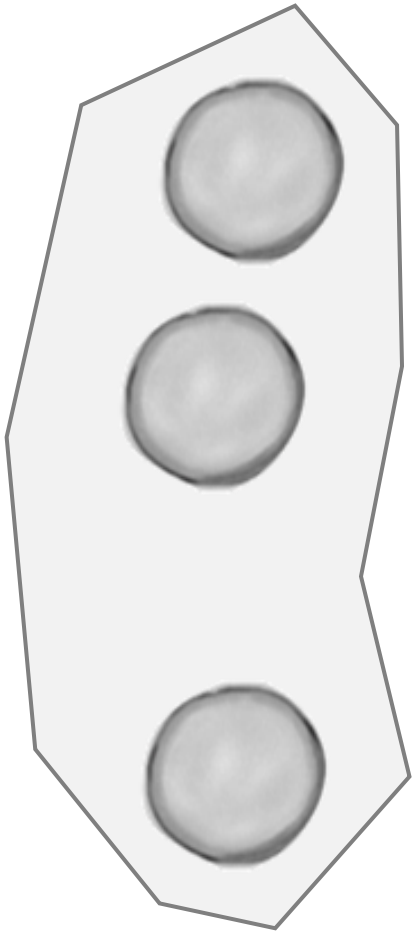
By coding to concrete implementations we have no way to add or remove other display elements without making changes to the program.



Observer Pattern



Subject
Object



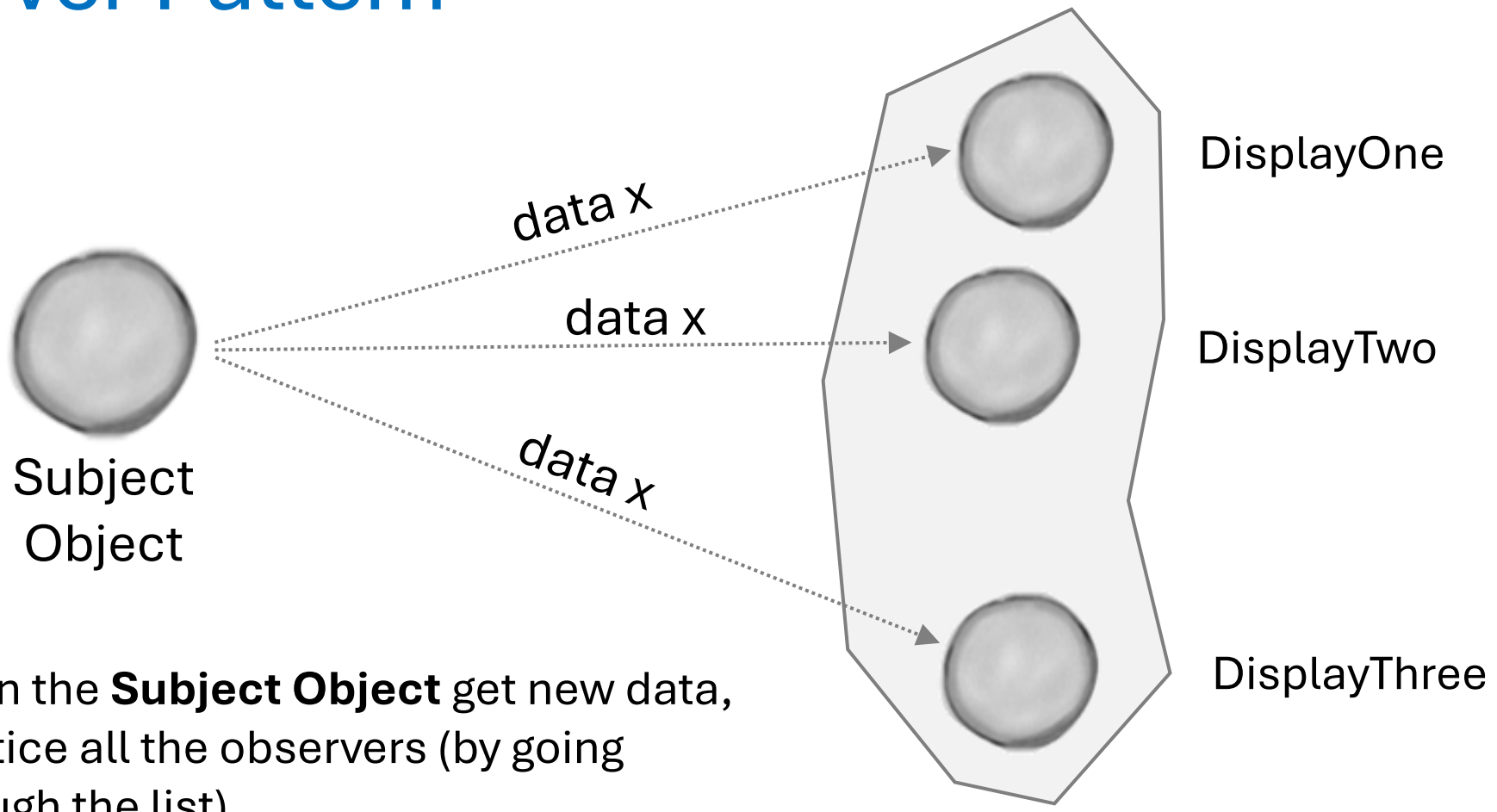
DisplayOne

DisplayTwo

DisplayThree

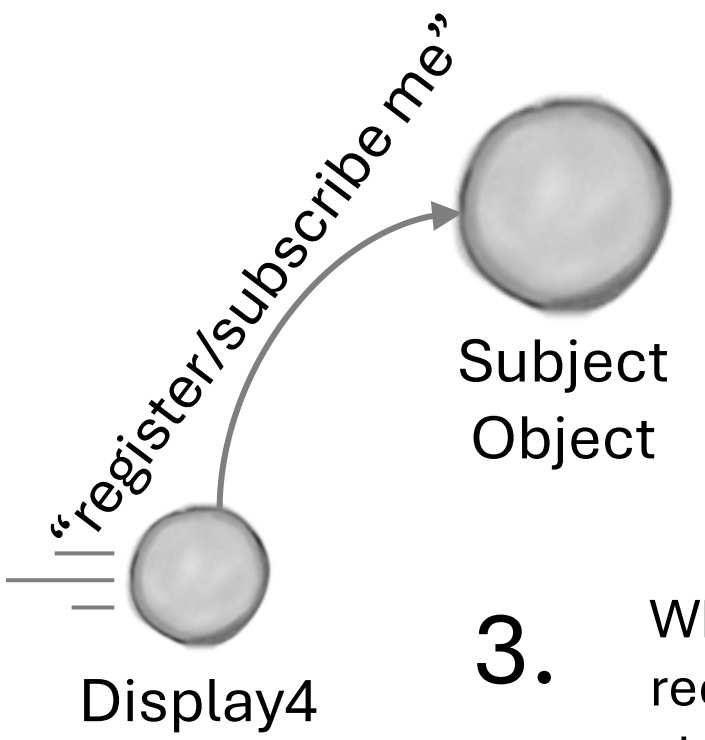
1. A Subject Object (e.g. our **WeatherData** object) maintains a list of observer (e.g. our Display objects).

Observer Pattern

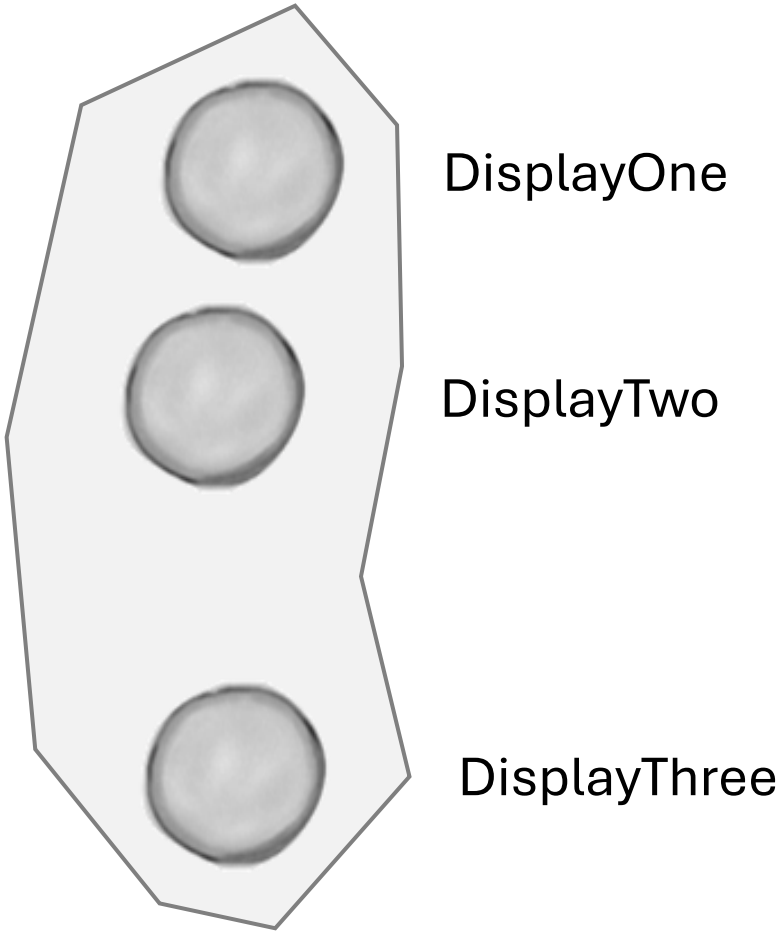


2. When the **Subject Object** get new data, it notice all the observers (by going through the list).

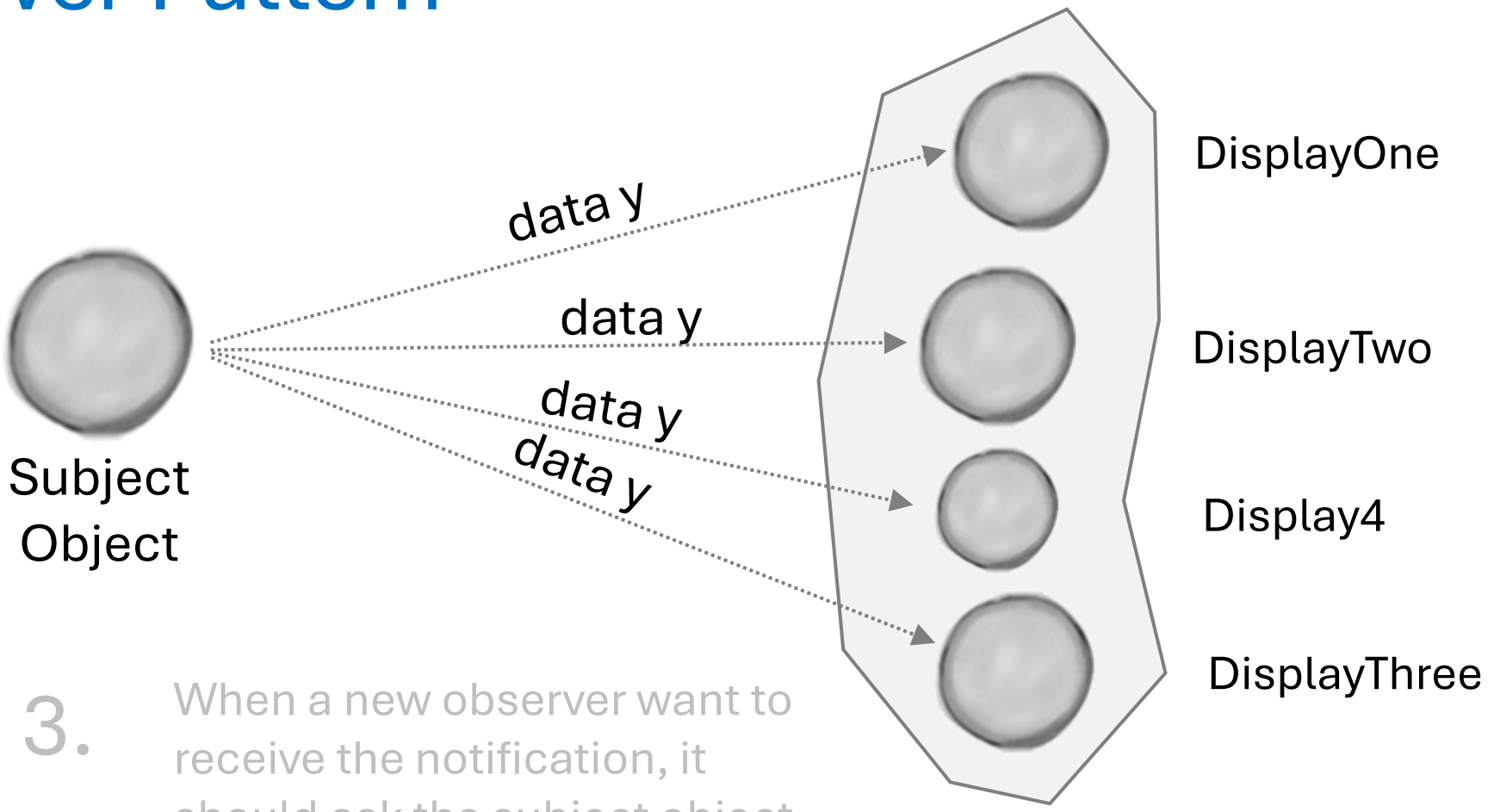
Observer Pattern



3. When a new observer want to receive the notification, it should ask the subject object to add itself into the list.

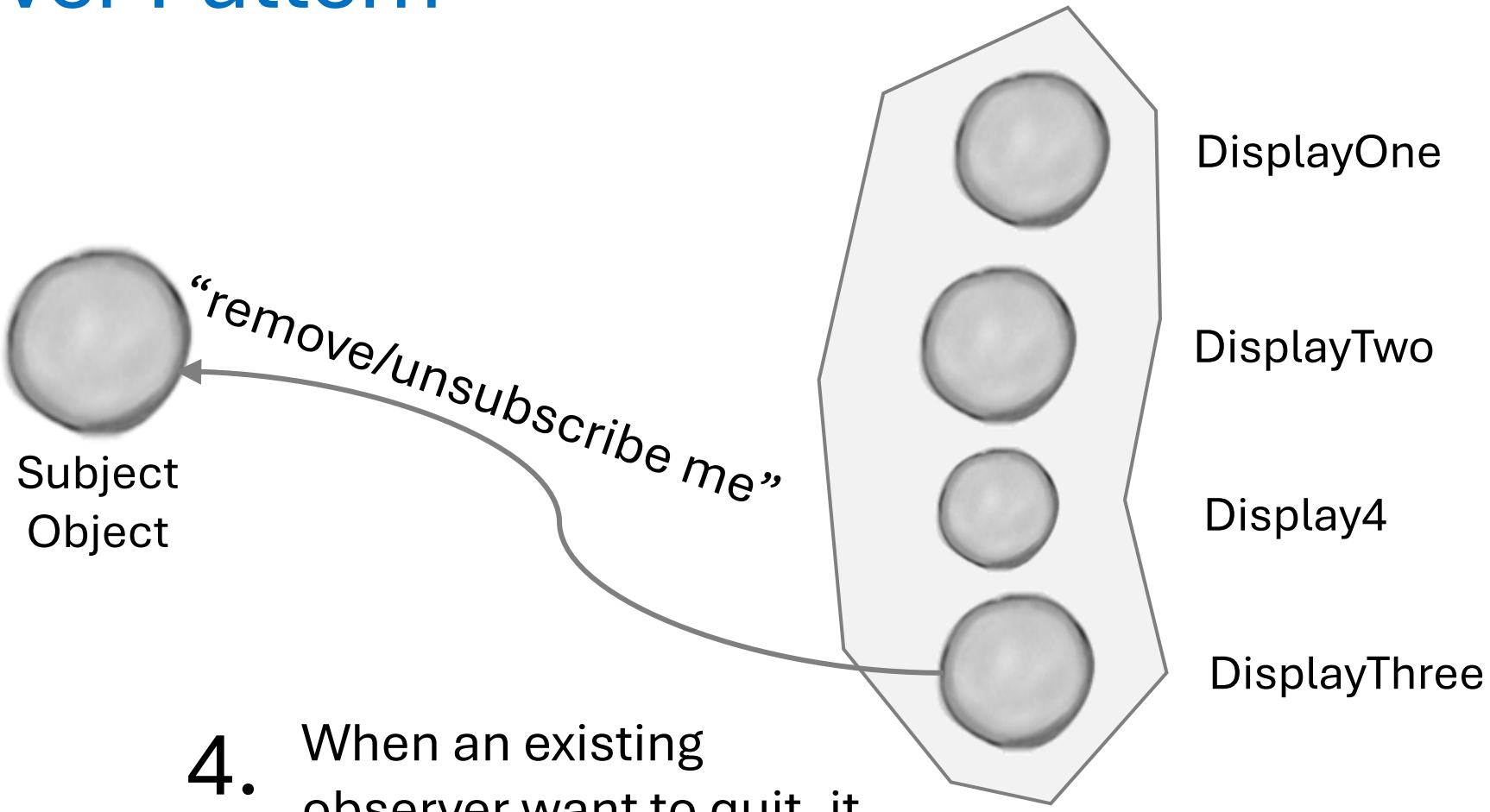


Observer Pattern



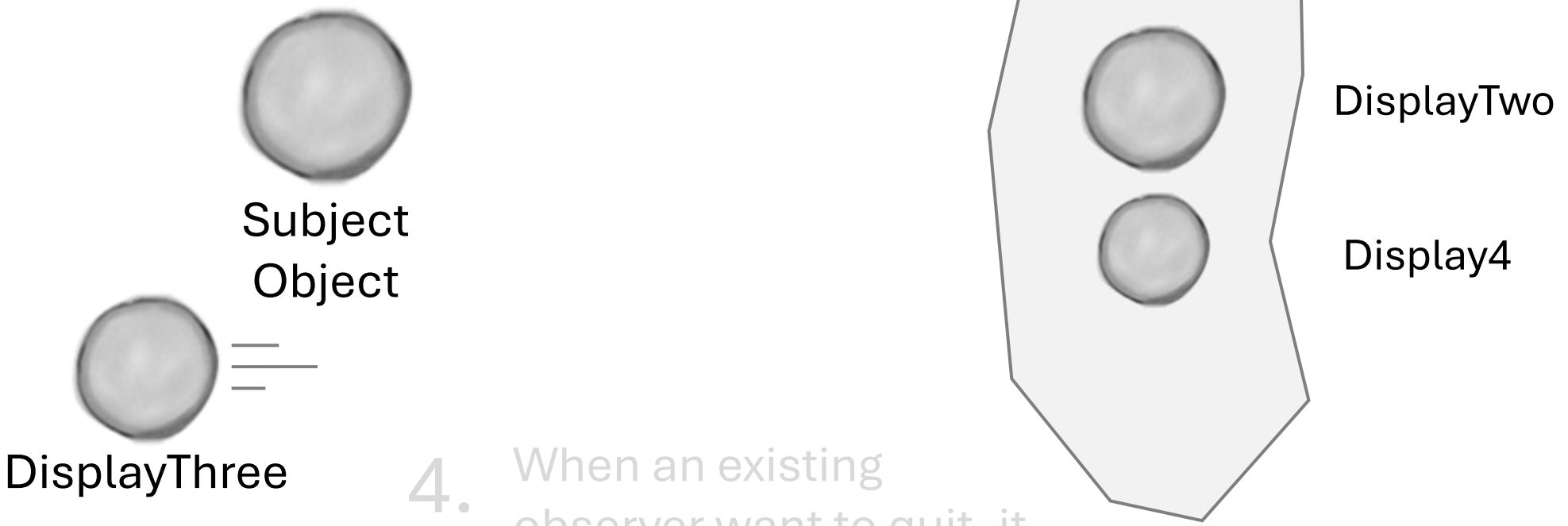
3. When a new observer want to receive the notification, it should ask the subject object to add itself into the list.

Observer Pattern



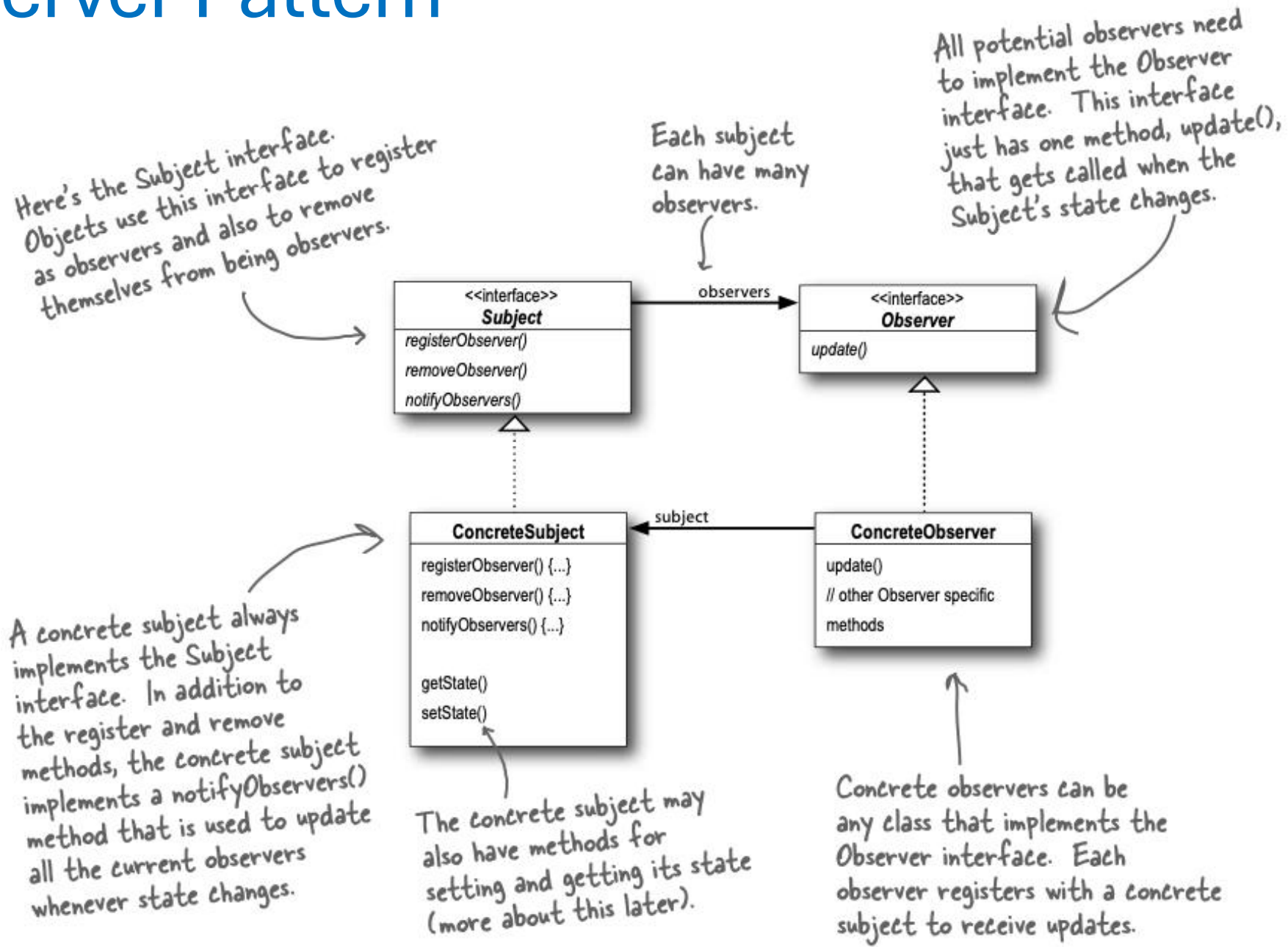
4. When an existing observer want to quit, it asks the **Subject** to remove it.

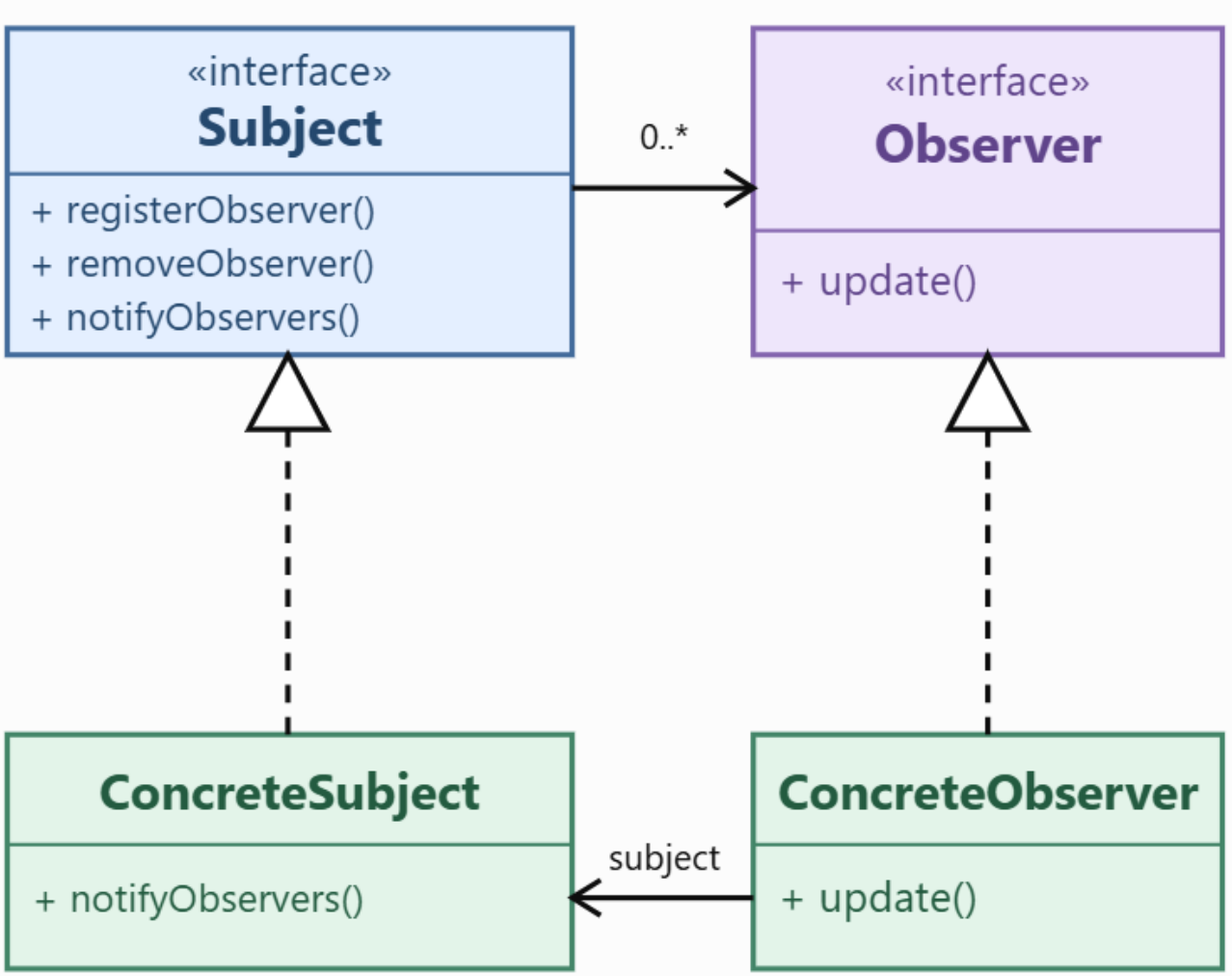
Observer Pattern



4. When an existing observer want to quit, it asks the **Subject** to remove it.

Observer Pattern





1. **The subject's state changes.**

   `channel.uploadVideo("New Tutorial")`

2. **The subject calls `notifyObservers()`.**

   It loops through its registered observers.

3. **Each observer receives `update()`.**

   `subscriber.update()`

4. **Observers react to the notification.**

   Each subscriber displays a new-video notification.

Mock quiz

In which of the following situations would you use the Observer design pattern?

A. When an abstraction has two aspects, one dependent on the other.
B. When a change to one object requires changing others, and you don't know how many objects need to be changed.
C. When an object should be able to notify other objects without making assumptions about who these objects are.
D. All of the above.

Observer Pattern-- Conclusion

In the Weather Monitoring app example, we identified that the changing part is to add new displays. Moreover, this caused the changing of the code of the message notification,.

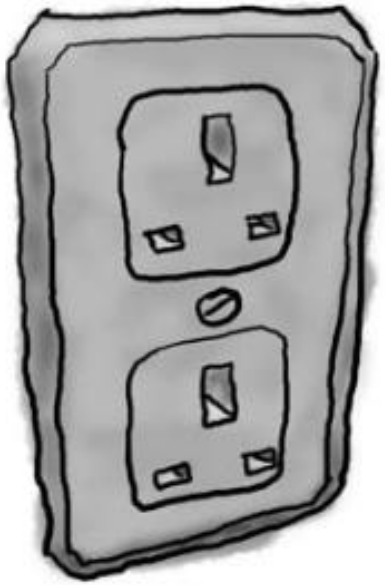
So we separate the sending message and the removing message behaviours from the WeatherStation and WeatherData classes, and create a new set of interfaces to represent each behaviour.

Adapter Pattern



Real world adapters

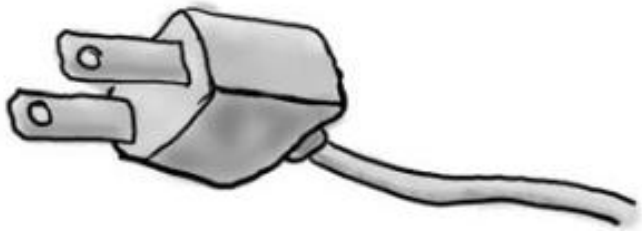
You'll have no trouble understanding what an OO adapter is because the real world is full of them.



UK Wall Outlet



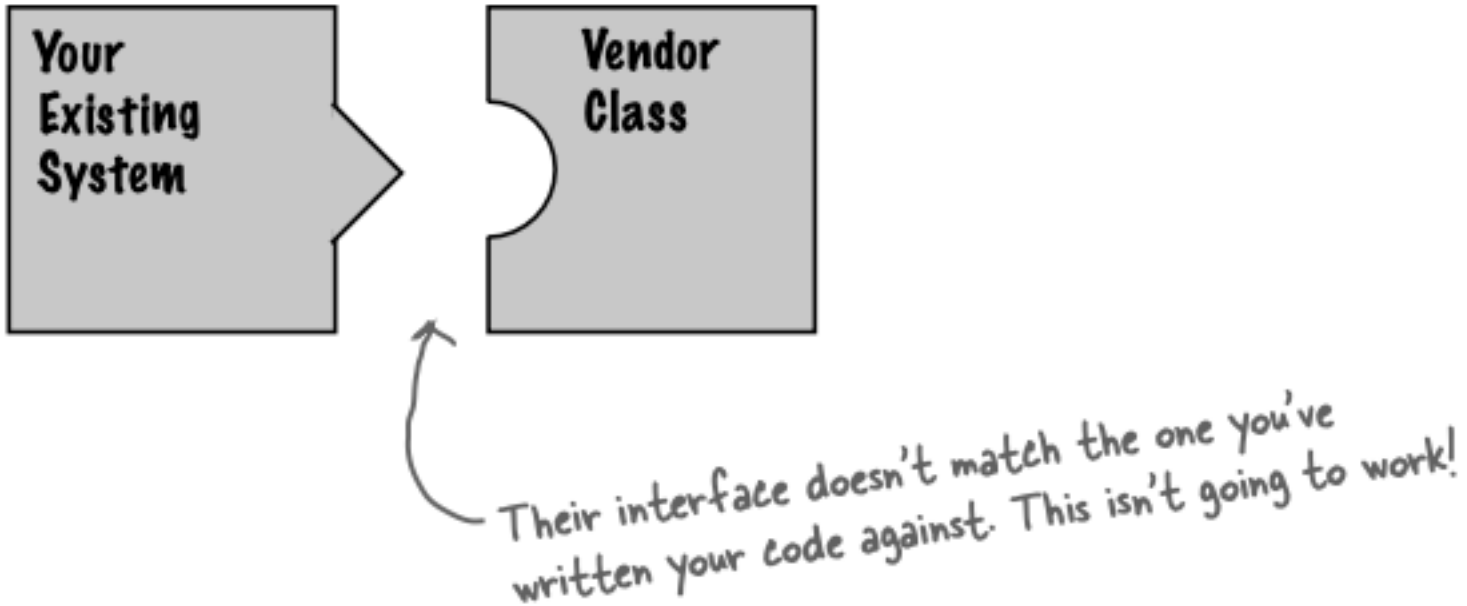
Power Adapter



Chinese Plug

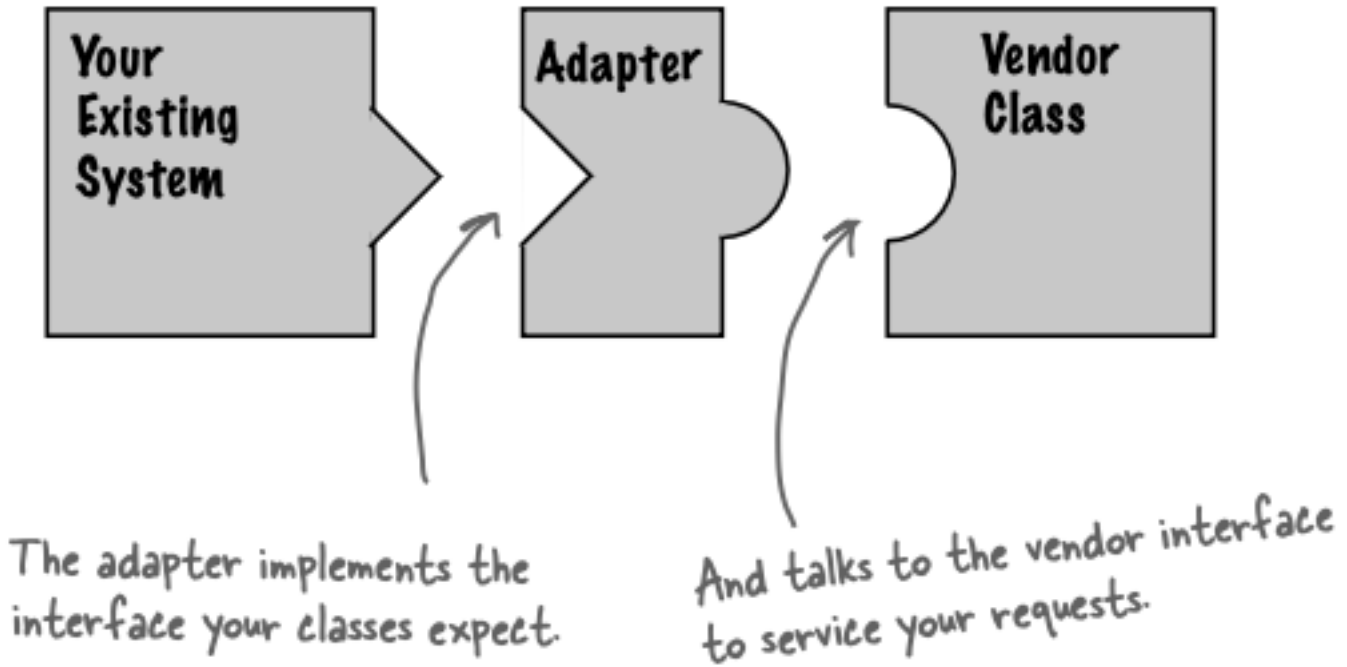
Object oriented adapters

Say you've got an existing software system that you need to work a new vendor class library into, but the new vendor designed their interfaces differently than the last vendor:



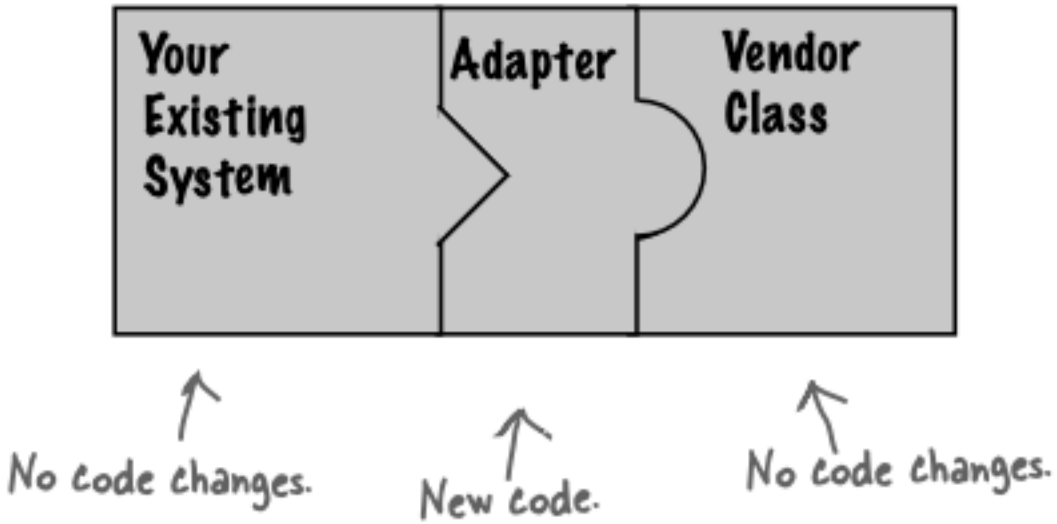
Object oriented adapters

However, you don’t want to solve the problem by changing your existing code (and you can’t change the vendor’s code). So, you can write a class that adapts the new vendor interface into the one you’re expecting.



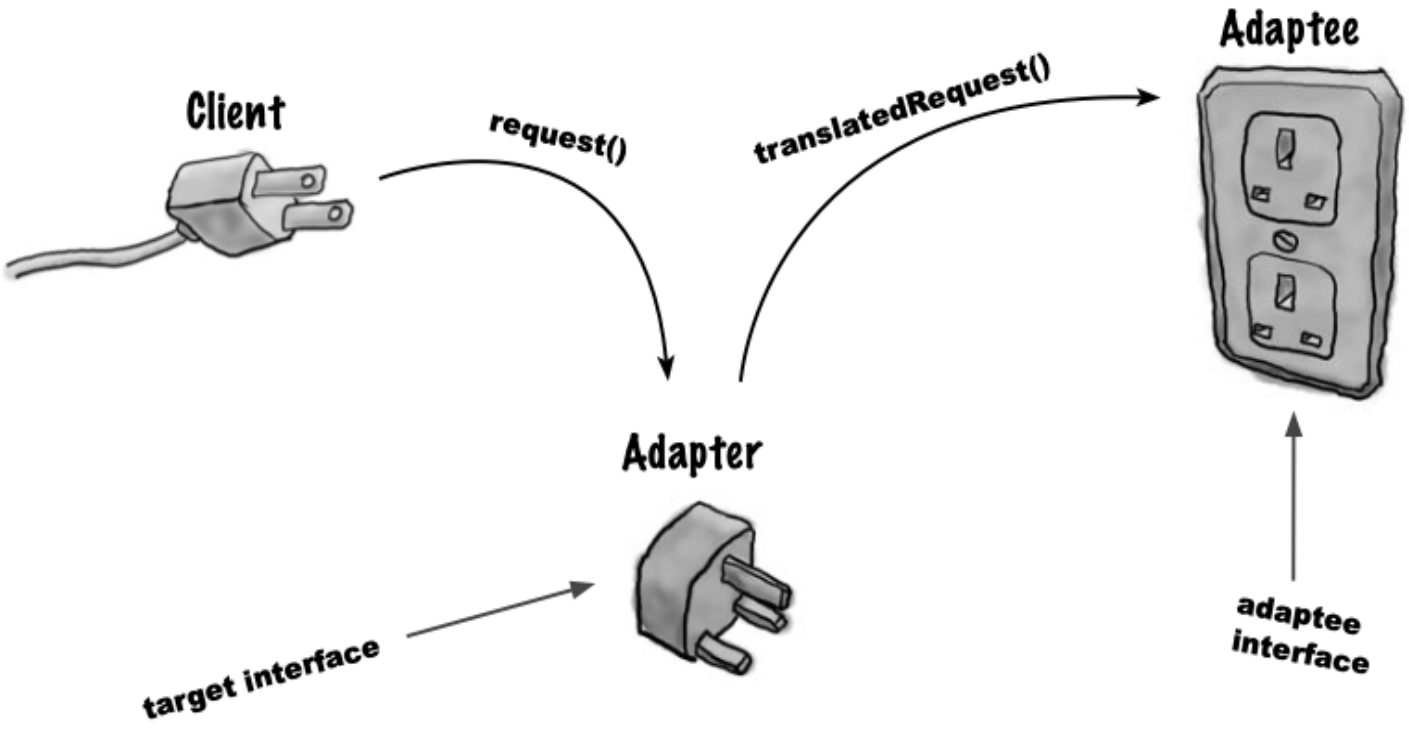
Object oriented adapters

The adapter acts as the middleman by receiving requests from the client and converting them into requests that make sense on the vendor classes.



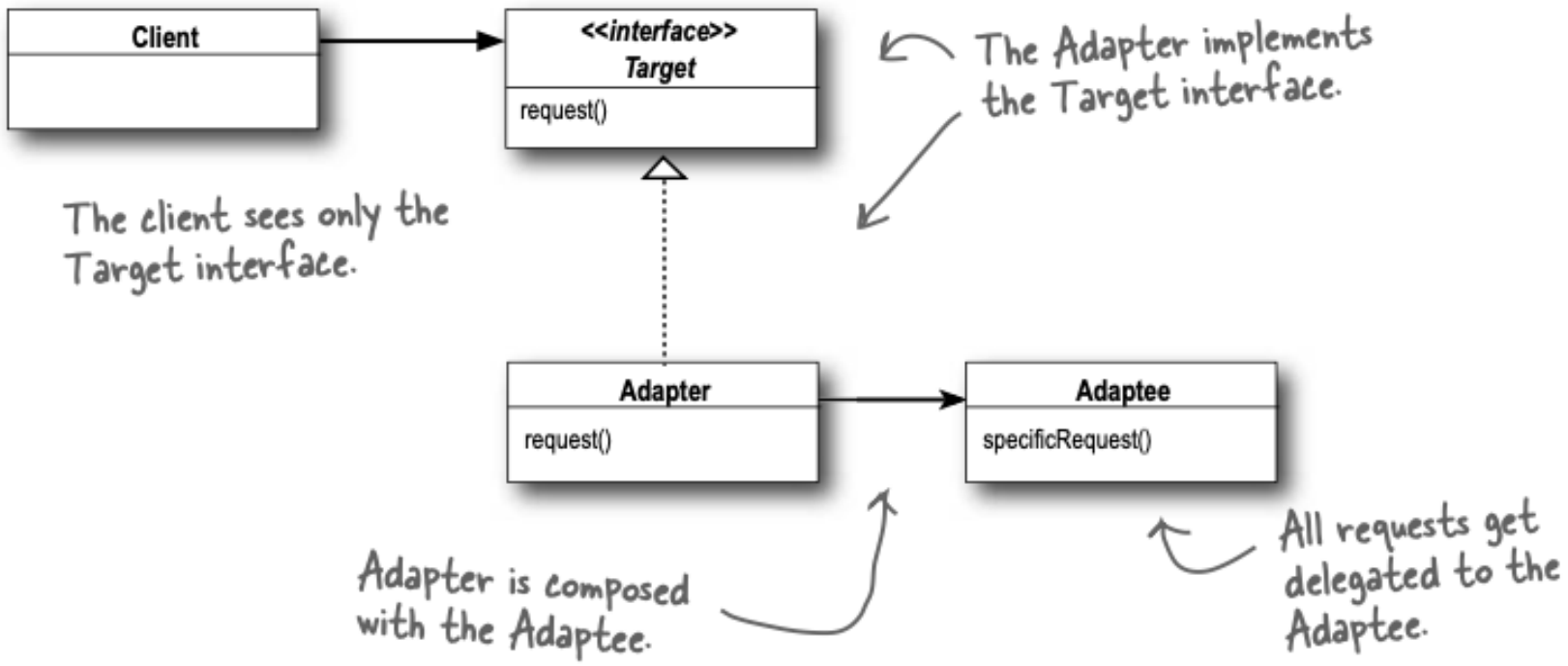
Adapter Pattern

1. The client makes a request to the adapter by calling a method on it using the target interface.
2. The adapter translates the request into one or more calls on the adaptee using the adaptee interface.
3. The client receives the results of the call and never knows there is an adapter doing the translation.



Adapter Pattern

The Adapter Pattern converts the interface of a class into another interface the clients expect. Adapter lets classes work together that couldn't otherwise because of incompatible interfaces.



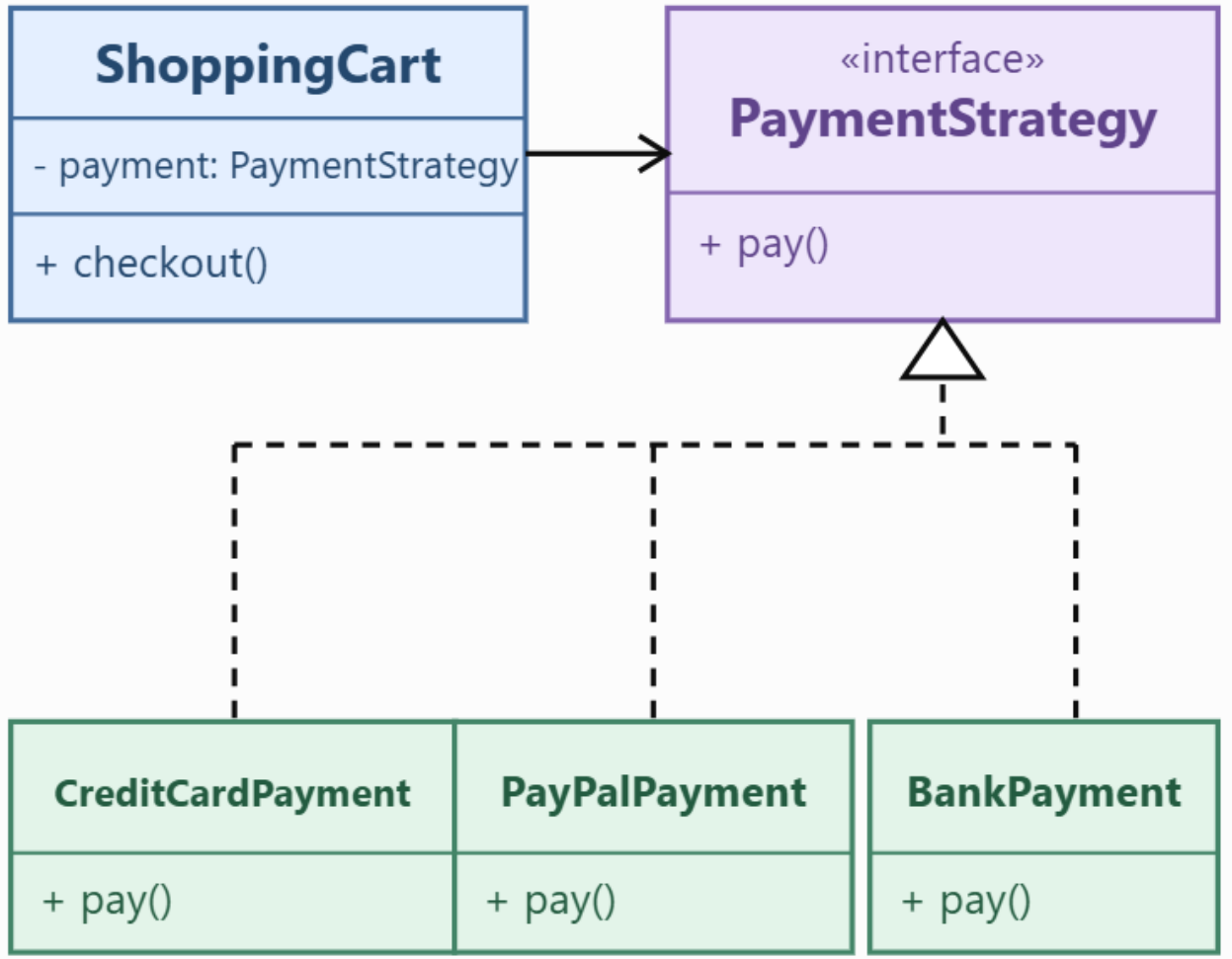
Strategy Design Pattern



Strategy Design Pattern

Define different ways of performing a task, put each way into a separate class, and allow the program to **switch between them without changing the main class**.

Example.
Payment System



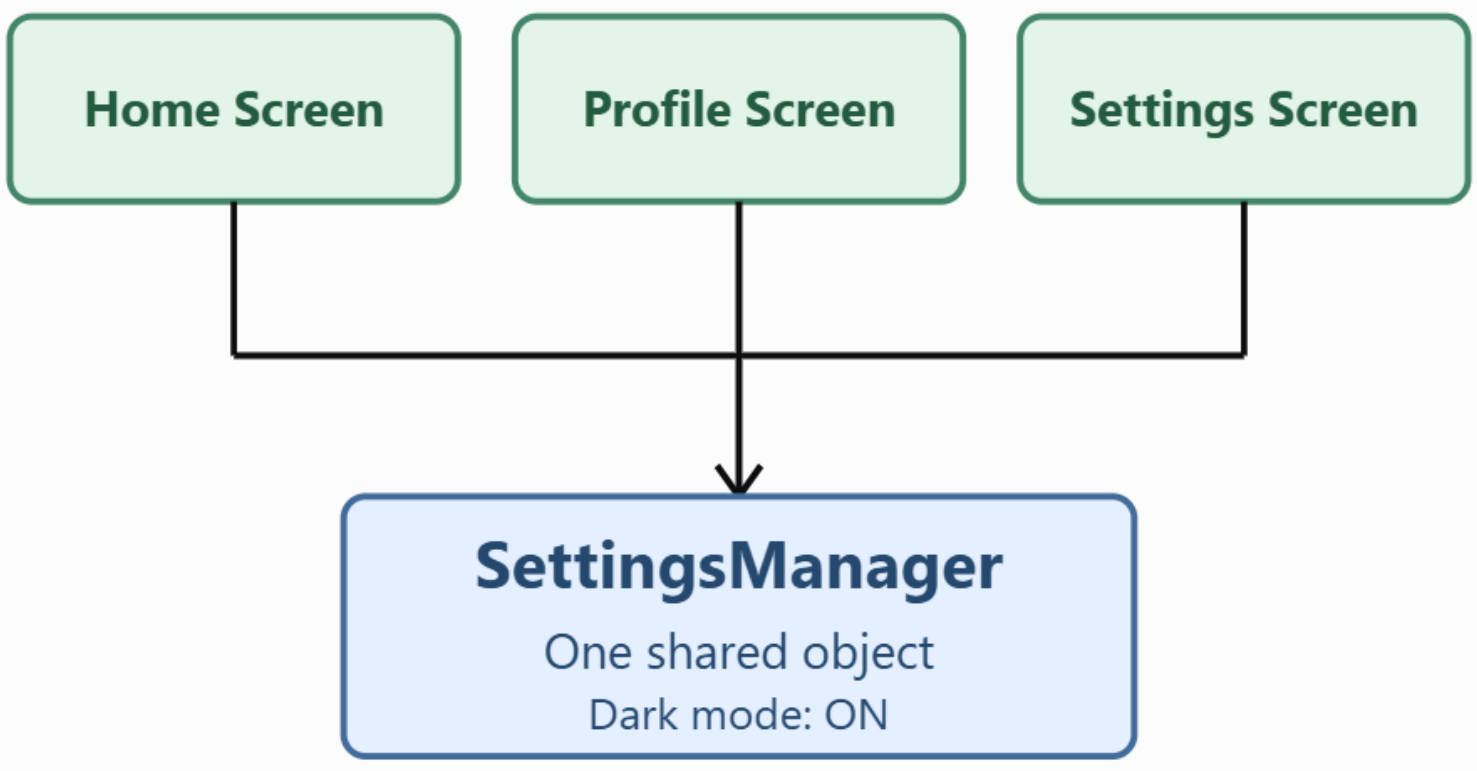
Singleton Pattern



Singleton Pattern

Ensure that a class has **only one instance (object)** and provide a global way to access that instance.

Example.
App Settings



Strategy Design Pattern

Ensure that a class has **only one instance (object)** and provide a global way to access that instance.

Example.
App Settings

